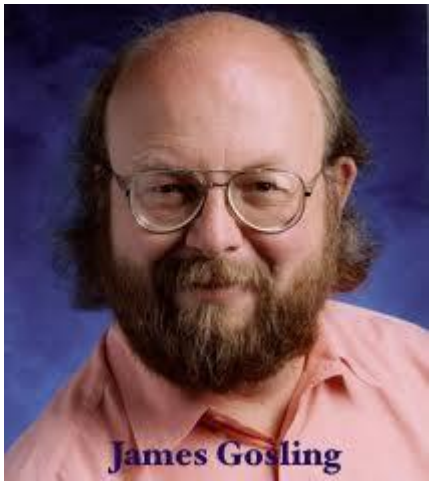
Unit=1

The Origins of Java

The history of Java is very interesting. Java was originally designed for interactive television, but it was too advanced technology for the digital cable television industry at the time. The history of Java starts with the Green Team. Java team members (also known as Green Team), initiated this project to develop a language for digital devices such as set-top boxes, televisions, etc. However, it was best suited for internet programming. Later, Java technology was incorporated by Netscape.

The principles for creating Java programming were "Simple, Robust, Portable, Platform-independent, Secured, High Performance, Multithreaded, Architecture Neutral, Object-Oriented, Interpreted, and Dynamic". Java was developed by James Gosling, who is known as the father of Java, in 1995. James Gosling and his team members started the project in the early '90s.



Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc. Following are given significant points that describe the history of Java.

1) **James Gosling**, **Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.

2) Initially it was designed for small, embedded systems in electronic appliances like set-top boxes.

3) Firstly, it was called **"Greentalk"** by James Gosling, and the file extension was .gt.

4) After that, it was called **Oak** and was developed as a part of the Green project.

Why Java was named as "Oak"?

5) **Why Oak?** Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.

6) In 1995, Oak was renamed as **"Java"** because it was already a trademark by Oak Technologies.

Why Java Programming named "Java"?

7) Why had they chose the name Java for Java language? The team gathered to choose a new name. The suggested words were "dynamic", "revolutionary", "Silk", "jolt", "DNA", etc. They wanted something that reflected the essence of the technology: revolutionary, dynamic, lively, cool, unique, and easy to spell, and fun to say.

According to James Gosling, "Java was one of the top choices along with **Silk**". Since Java was so unique, most of the team members preferred Java than other names.

8) Java is an island in Indonesia where the first coffee was produced (called Java coffee). It is a kind of espresso bean. Java name was chosen by James Gosling while having a cup of coffee nearby his office.

9) Notice that Java is just a name, not an acronym.

10) Initially developed by James Gosling at Sun Microsystems (which is now a subsidiary of Oracle Corporation) and released in 1995.

11) In 1995, Time magazine called **Java one of the Ten Best Products of 1995**.

12) JDK 1.0 was released on January 23, 1996. After the first release of Java, there have been many additional features added to the language. Now Java is being used in Windows applications, Web applications, enterprise applications, mobile applications, cards, etc. Each new version adds new features in Java.

Java Version History

Many java versions have been released till now. The current stable release of Java is Java SE 10.

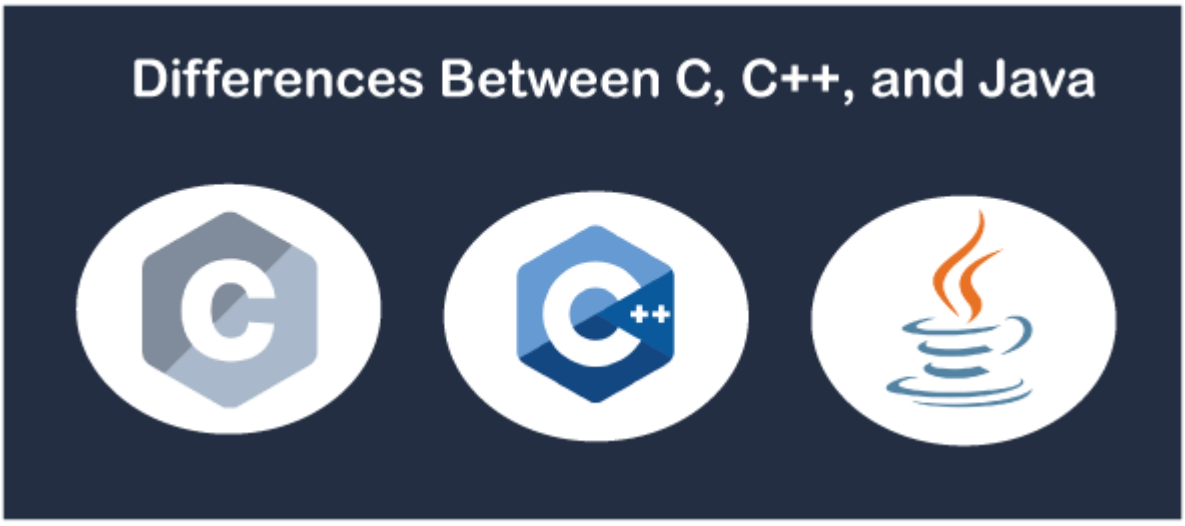
1. JDK Alpha and Beta (1995)
2. JDK 1.0 (23rd Jan 1996)
3. JDK 1.1 (19th Feb 1997)
4. J2SE 1.2 (8th Dec 1998)
5. J2SE 1.3 (8th May 2000)
6. J2SE 1.4 (6th Feb 2002)
7. J2SE 5.0 (30th Sep 2004)
8. Java SE 6 (11th Dec 2006)
9. Java SE 7 (28th July 2011)
10. Java SE 8 (18th Mar 2014)
11. Java SE 9 (21st Sep 2017)
12. Java SE 10 (20th Mar 2018)
13. Java SE 11 (September 2018)
14. Java SE 12 (March 2019)
15. Java SE 13 (September 2019)
16. Java SE 14 (Mar 2020)
17. Java SE 15 (September 2020)
18. Java SE 16 (Mar 2021)
19. Java SE 17 (September 2021)
20. Java SE 18 (to be released by March 2022)

Since Java SE 8 release, the Oracle corporation follows a pattern in which every even version is release in March month and an odd version released in September month.

Java's Lineage: C and C++

Difference Between C, C++, and Java

The programming languages **C**, **C++**, and **Java** are the most popular and widely used programming languages in the market. In this section, we will discuss the **differences between C, C++**, and **Java.** Apart from the similarities in syntax and that both are object-oriented, their design aim and philosophy are different.



C

C is a general-purpose, structured, procedural, and high-level programming language developed by **Dennis MacAlistair Ritchie** in 1972 at **Bell Laboratories**. The successor of the C language was CPL (Combined Programming Language). It is mainly used for system programming such as to develop the operating system, drivers, compilers, etc.

The best-known example of the operating system that was developed using C language is Unix and Linux.

Features of C Language

- Machine independent and portable
- Modern Control Flow and Structure
- Rich set of operators
- Simple, Fast, and efficient
- Case-sensitive
- Low memory use
- Easily extendable
- Statically-typed

C++

C++ is an object-oriented, general-purpose, programming language developed by **Bjarne Stroustrup** at Bell Labs in 1979. It is based on C language or we can say that it is an extension of C language. It is used to develop high-performance applications.

Features of C++ Language

- Case-sensitive
- Compiler based
- Platform-independent
- Portability
- Dynamic memory allocation

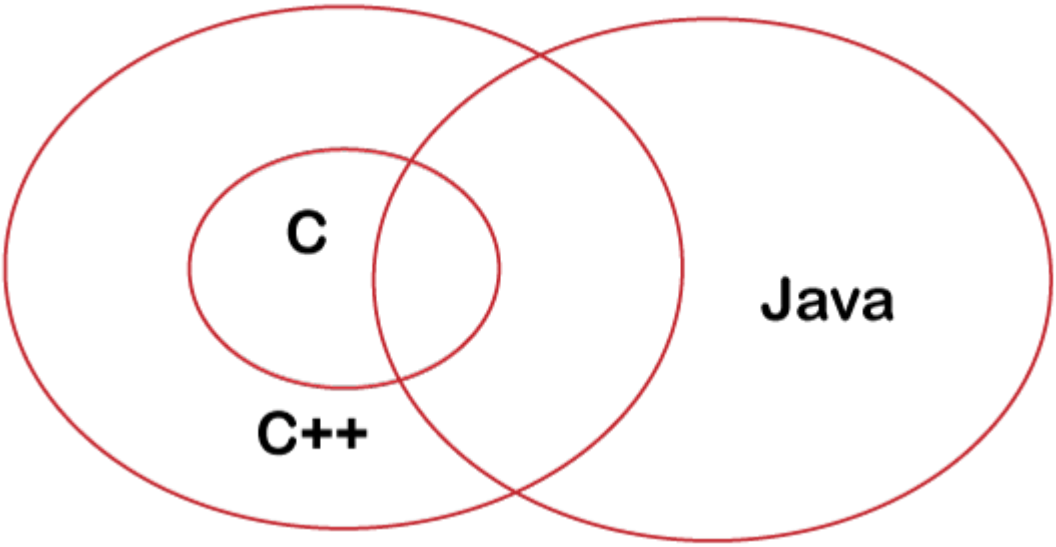
Java

Java is also an object-oriented, class-based, static, strong, robust, safe, and high-level programming language. It was developed by **James Gosling** in 1995. It is bot compiled and interpreted. It is used to develop enterprise, mobile, and web-based applications.

Features of Java

- Object-oriented
- Architecture-neutral
- Platform independent
- Dynamic and Distributed
- Robust
- Secure
- Multithreaded

The following figure demonstrates that C++ is based on the C language and Java is based on the C++ and C language.



How Java Impacted the Internet

1. Platform Independence

Java's "Write Once, Run Anywhere" (WORA) philosophy revolutionized web development. Since Java programs run on the Java Virtual Machine (JVM), they can be executed on any device with a JVM installed, regardless of the underlying operating system. This flexibility made it easier to deploy web applications across various platforms, contributing to the growth of internet applications.

2. Java Applets

In the early days of the web, Java Applets were widely used to embed dynamic content in web pages. These small applications, which could be executed in browsers, allowed developers to create interactive, rich-media content. While applets have become obsolete due to security concerns and the rise of alternatives like JavaScript, they were groundbreaking in enabling interactive web experiences in the late 1990s and early 2000s.

3. Server-Side Technologies

Java's influence on the backend side of web development is immense:

- **Java Servlets and JSP (JavaServer Pages)**: These technologies allowed developers to create dynamic web pages and server-side applications. Java became a popular choice for developing scalable, robust web applications.
- **Enterprise JavaBeans (EJB)**: These components enabled distributed, transactional, and scalable applications, which were essential for enterprise-level web services.
- **Spring Framework**: Java's Spring framework further boosted the development of server-side applications with features like dependency injection and aspect-oriented programming. It became one of the most popular frameworks for building web applications.

4. Mobile Internet and Android

Java's role in the mobile internet is substantial due to its use in Android development. Android, the world's most popular mobile operating system, relies heavily on Java (although now Kotlin is also widely used). As mobile internet usage exploded, Java played a critical role in enabling the development of millions of mobile apps, shaping the way people access and interact with the web.

5. Web Security

Java introduced a comprehensive security model, including features like bytecode verification, sandboxing, and cryptography. This allowed developers to build secure web applications, critical for e-commerce and financial services that flourished with the rise of the internet.

6. E-commerce and Enterprise Web Applications

With the help of frameworks like Java EE (Enterprise Edition), Java has been a cornerstone for building large-scale e-commerce platforms and enterprise web applications. Companies like Amazon and eBay have used Java to build scalable, reliable, and secure web-based systems.

7. Microservices and Cloud Computing

Java continues to impact modern web architecture through microservices and cloud-based systems. Frameworks like **Spring Boot** are widely used to build microservices, which are crucial in today's cloud computing environments. Java's performance, scalability, and compatibility with cloud platforms like AWS, Google Cloud, and Microsoft Azure ensure its continued relevance in internet infrastructure.

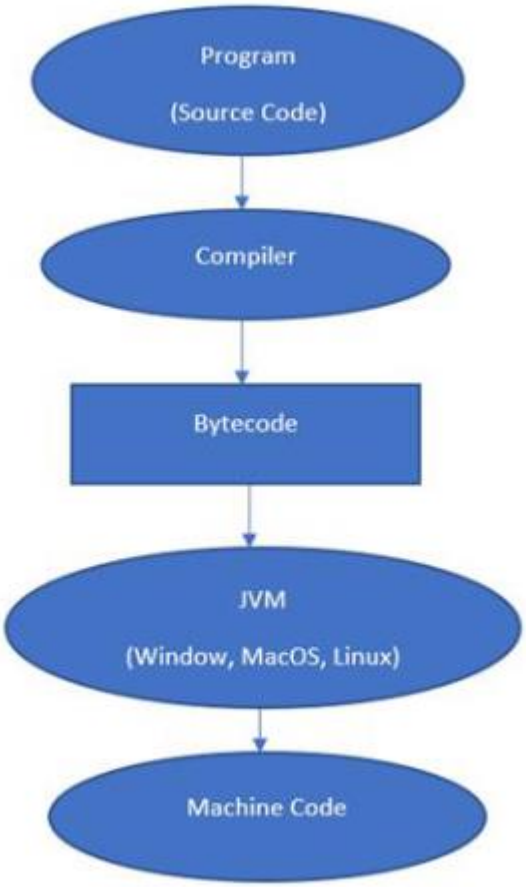
8. APIs and Web Services

Java has contributed significantly to the development of web services using protocols like **SOAP** and **REST**. Java APIs for RESTful Web Services (JAX-RS) make it easier to integrate and communicate between different web applications, enhancing the interconnectedness of the internet.

Java's Magic: The Bytecode

Byte Code can be defined as an intermediate code generated by the compiler after the compilation of source code(JAVA Program). This intermediate code makes Java a platform-independent language.

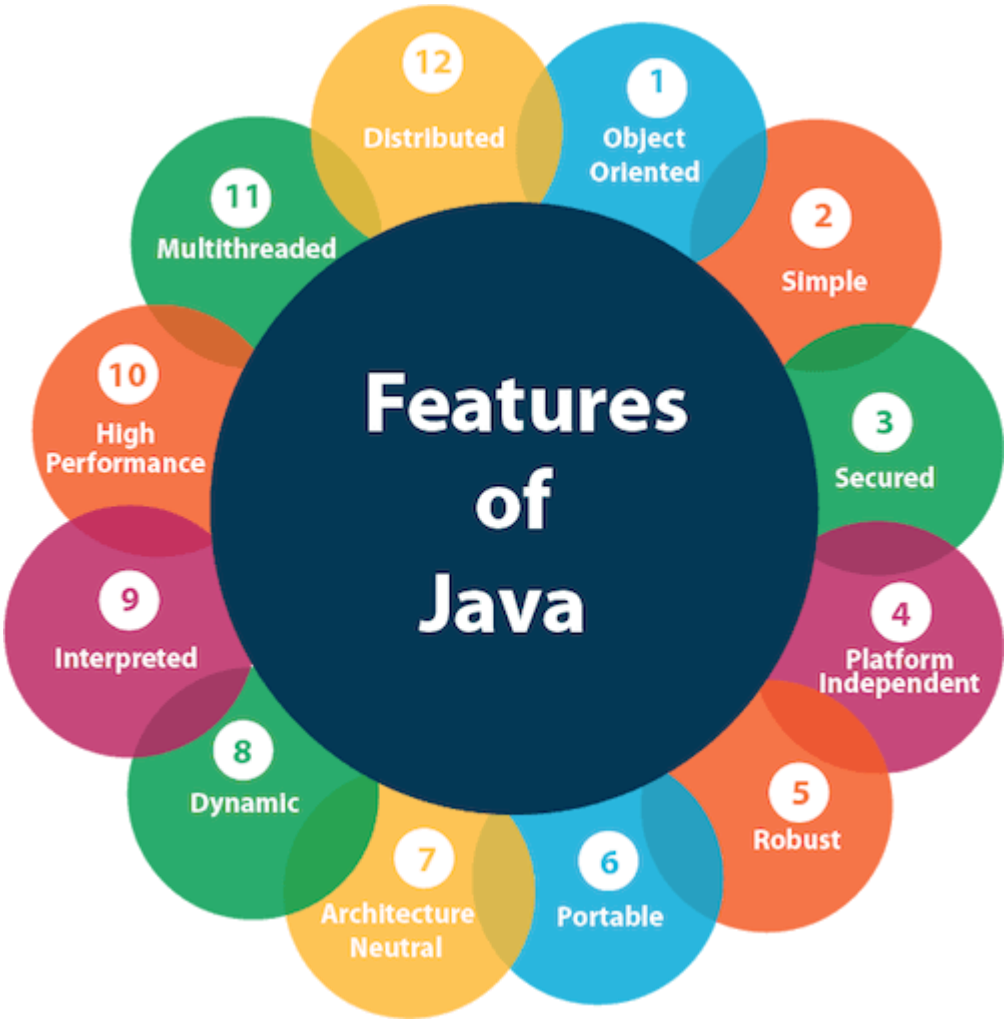
How is Byte Code generated?
Compiler converts the source code or the Java program into the Byte Code(or machine code), and secondly, the Interpreter executes the byte code on the system. The Interpreter can also be called JVM(Java Virtual Machine). The byte code is the common piece between the compiler(which creates it) and the Interpreter (which runs it).



The Java Buzzwords

The primary objective of Java programming language creation was to make it portable, simple and secure programming language. Apart from this, there are also some excellent features which play an important role in the popularity of this language. The features of Java are also known as Java buzzwords.

A list of the most important features of the Java language is given below.



1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High Performance
10. Multithreaded
11. Distributed
12. Dynamic

Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystem, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

Object-oriented

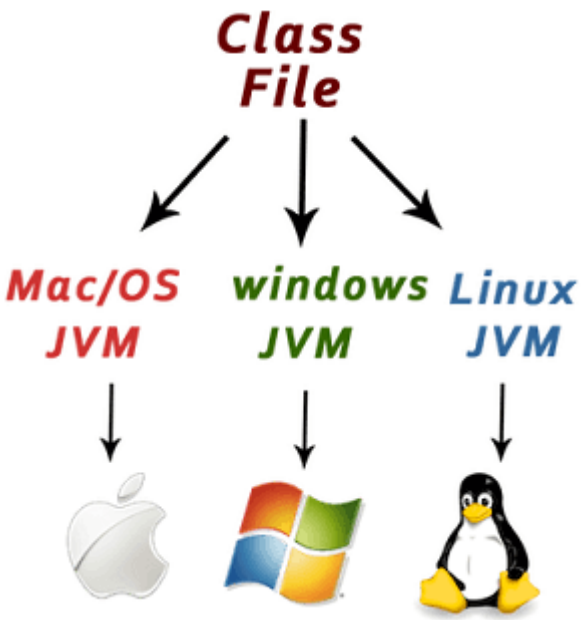
Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.

Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are:

1. Object
2. Class
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation

Platform Independent



Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.

There are two types of platforms software-based and hardware-based. Java provides a software-based platform.

The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on top of other hardware-based platforms. It has two components:

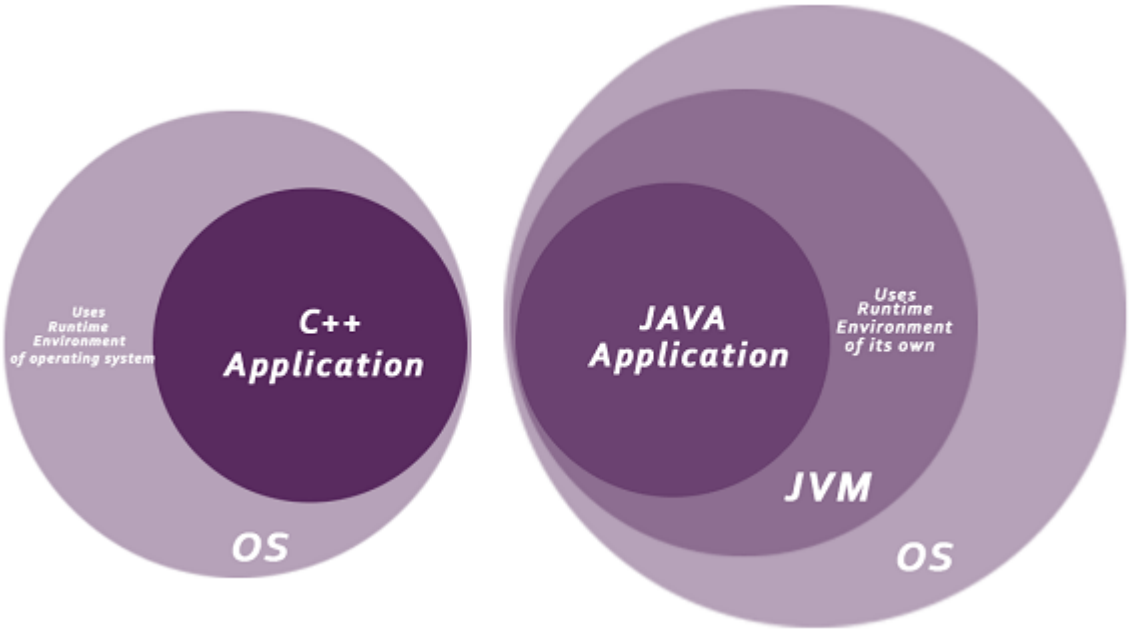
1. Runtime Environment
2. API(Application Programming Interface)

Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. Java code is compiled by the compiler and converted into bytecode. This bytecode is a platform-independent code because it can be run on multiple platforms, i.e., Write Once and Run Anywhere (WORA).

Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**



- **Classloader:** Classloader in Java is a part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier:** It checks the code fragments for illegal code that can violate access rights to objects.
- **Security Manager:** It determines what resources a class can access such as reading and writing to the local disk.

Java language provides these securities by default. Some security can also be provided by an application developer explicitly through SSL, JAAS, Cryptography, etc.

Robust

The English mining of Robust is strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

Architecture-neutral

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

Portable

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

High-performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

Distributed

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Multi-threaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

Dynamic

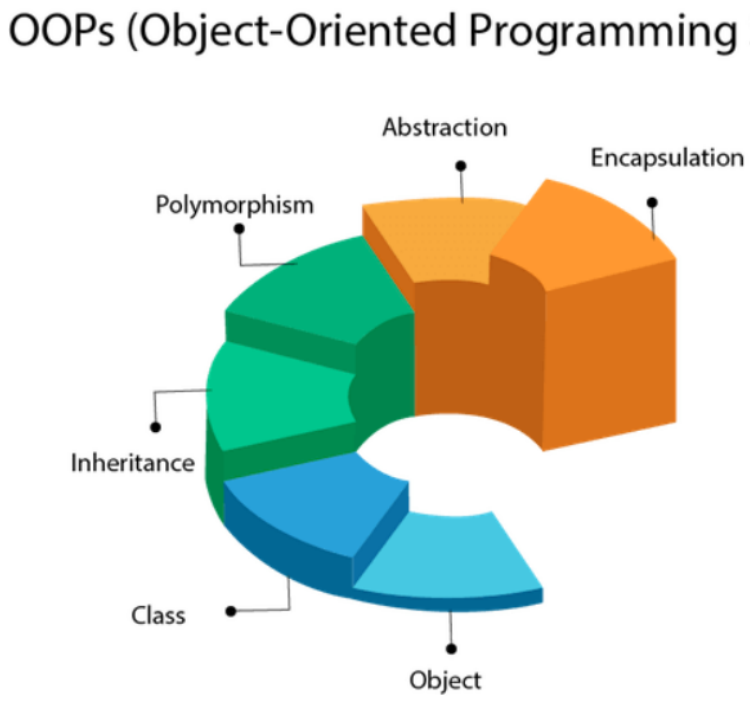
Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

Java supports dynamic compilation and automatic memory management (garbage collection).

OOPs (Object-Oriented Programming System)

Object means a real-world entity such as a pen, chair, table, computer, watch, etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation



Object

Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An Object can be defined as an instance of a class. An object contains an address and takes up some space in memory. Objects can communicate without knowing the details of

each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.

Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

Class

Collection of objects is called class. It is a logical entity.

A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

Inheritance

When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism

Polymorphism

If one task is performed in different ways, it is known as polymorphism. For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.

In Java, we use method overloading and method overriding to achieve polymorphism.

Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example phone call, we don't know the internal processing.

In Java, we use abstract class and interface to achieve abstraction.

Encapsulation

Binding (or wrapping) code and data together into a single unit are known as encapsulation. For example, a capsule, it is wrapped with different medicines.

A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

Inheritance

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

Inheritance represents the **IS-A relationship** which is also known as a parent-child relationship.

Terms used in Inheritance

- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
  //methods and fields
}
```

The **extends keyword** indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

Example

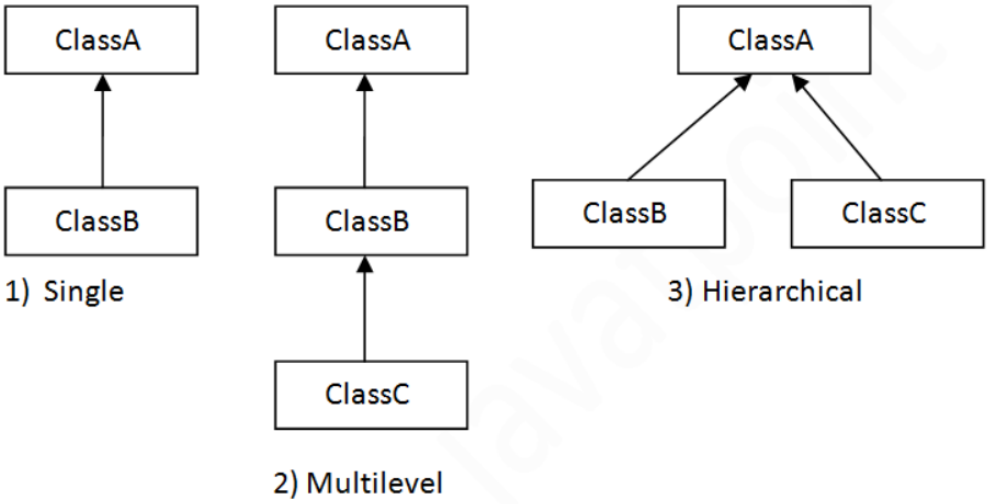
```
.        class Employee{
.         float salary=40000;
.        }
.        class Programmer extends Employee{
.         int bonus=10000;
.         public static void main(String args[]){
.           Programmer p=new Programmer();
.           System.out.println("Programmer salary is:"+p.salary);
.           System.out.println("Bonus of Programmer is:"+p.bonus);
.        }
.        }
```

Output:

```
Programmer salary is:40000.0
Bonus of programmer is:10000
```

Types of inheritance in java

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.



Single Inheritance Example

When a class inherits another class, it is known as a single inheritance. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

Example

```
.       class Animal{
.       void eat(){System.out.println("eating...");}
.       }
.       class Dog extends Animal{
.       void bark(){System.out.println("barking...");}
.       }
.       class TestInheritance{
.       public static void main(String args[]){
.       Dog d=new Dog();
.       d.bark();
.       d.eat();
.       }}
```

Output

```
barking...
eating...
```

Multilevel Inheritance Example

When there is a chain of inheritance, it is known as multilevel inheritance. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:

```
weeping...
barking...
```

```
eating...
```

Hierarchical Inheritance Example

When two or more classes inherits a single class, it is known as hierarchical inheritance. In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
}}
```

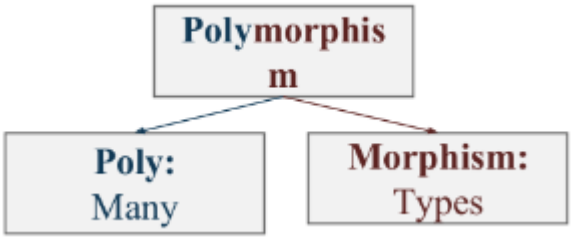
Output:

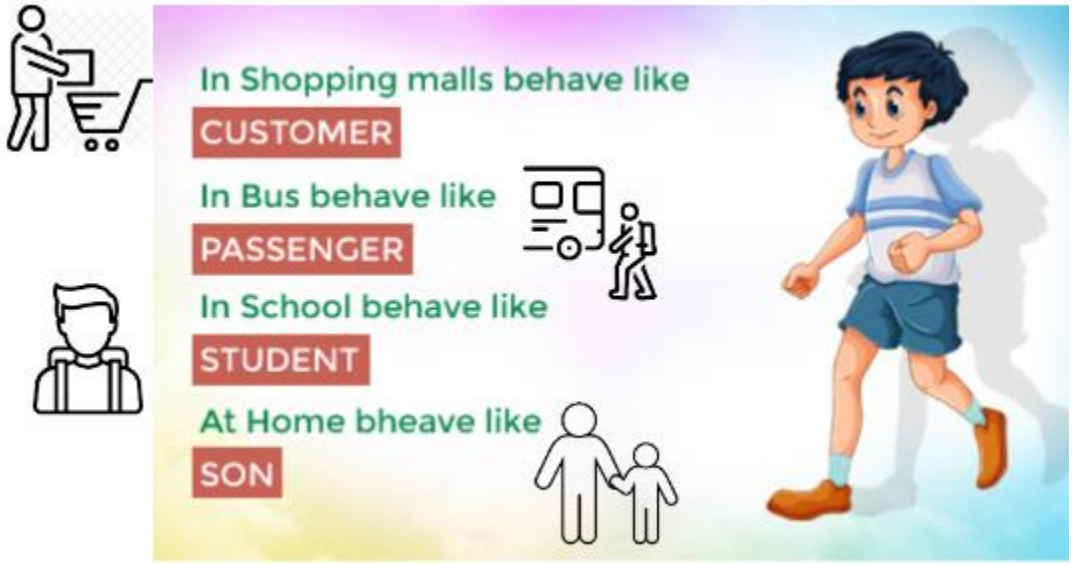
```
meowing...
eating..
```

Polymorphism

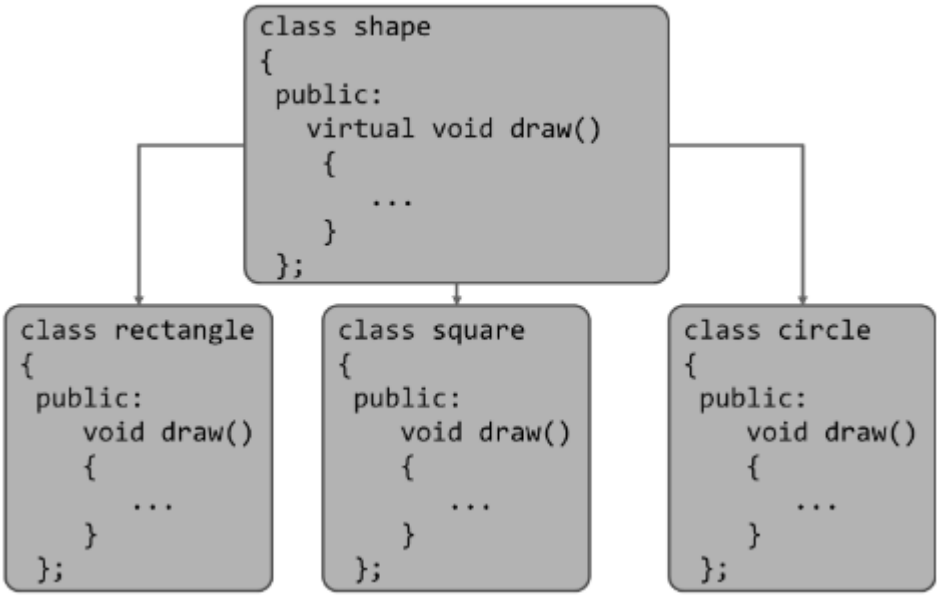
Polymorphism: It is a Greek term means, "One name many Forms"

Most important concept of object oriented programming .In OOP, Polymorphism is the ability of an object to take on many forms.





Polymorphism is the method in an object-oriented programming language that does different things depending on the class of the object which calls it. Polymorphism can be implemented using the concept of overloading and overriding.



Polymorphism: Advantages

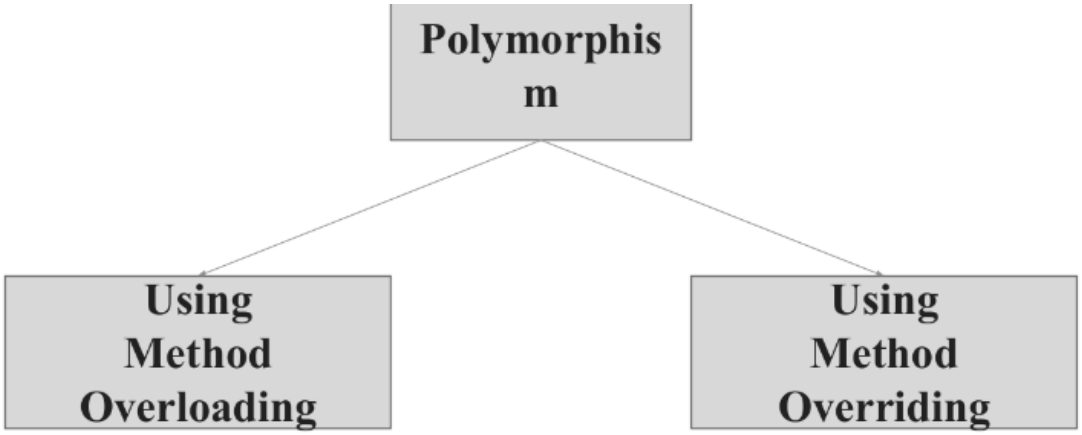
1.Single variable can be used to store multiple data types.

2.Easy to debug the codes.

3.It allows to perform a single act in different ways.

4.Polymorphism allows the object to decide which form of the function to implement at compile-time (overloading) as well as run-time (overriding).

5.Reduces coupling, increases reusability and makes code easier to read.



1. Method Overloading:

Definition: When two or more methods are implemented that share same name but different

parameter(s), the methods are said to be overloaded, and the process is referred to as method

overloading

Method overloading is one of the ways that Java implements polymorphism.

When an overloaded method is invoked, Java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to actually call.

Example:

public void draw()

public void draw(int height, int width)

public void draw(int radius)

Thus, overloaded methods must differ in the type and/or number of their parameters.

While in overloaded methods with different return types and same name & parameter are not allowed ,as the return type alone is insufficient for the compiler to distinguish two versions of a method.

Example

```java
class Addition{
int i,j,k;
 void add(int a){
 i=a;
 System.out.println("add i="+i);
  }
 void add(int a,int b){\\overloaded add()
 i=a;
 j=b;
 System.out.println("add i+j="+(i+j));
  }
 void add(int a,int b,int c){\\overloaded add()
 i=a;
 j=b;
 k=c;
 System.out.println("add i+j+k="+(i+j+k));
  }
}
```

```java
 class OverloadDemo{
public static void
main(String[] args){
  Addition a1= new Addition();
  //call all versions of add()
 a1.add(20);
 a1.add(30,50);
 a1.add(10,30,60);
 }
}
```

Output

```
add i=20
add i+j=80
add i+j+k=100
```

Method Overriding

In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass.

Definition: If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.

Example:

```
class Shape {

   void draw() {
      System.out.println("Draw Shape");
   }

}

class Circle extends Shape {

   @Override
   void draw() {
      System.out.println("Draw Circle");
   }

}

class Square extends Shape {
```

```
    @Override
    void draw() {
        System.out.println("Draw Square");
    }

}

public class OverrideDemo {

    public static void main(String[] args) {
        Circle c = new Circle();
        c.draw();  // Output: Draw Circle

        Square sq = new Square();
        sq.draw();  // Output: Draw Square
    }

}
```

Output

Draw Circle
Draw Square

Encapsulation

Encapsulation *is a process of wrapping code and data together into a single unit, for example, a capsule which is mixed of several medicines.*

We can create a fully encapsulated class in Java by making all the data members of the class private. Now we can use setter and getter methods to set and get the data in it.

Capsule

Example:

```
package com.javatpoint;

public class Student {
```

```
    // private data member
    private String name;

    // getter method for name
    public String getName() {
        return name;
    }

    // setter method for name
    public void setName(String name) {
        this.name = name;
    }
}

class TestStudent {
    public static void main(String[] args) {
        // Creating an instance of Student
        Student student = new Student();

        // Setting the value of the name
        student.setName("John");

        // Getting the value of the name
        System.out.println("Student's Name: " + student.getName());
    }
}
```

Output:
Student's Name: John

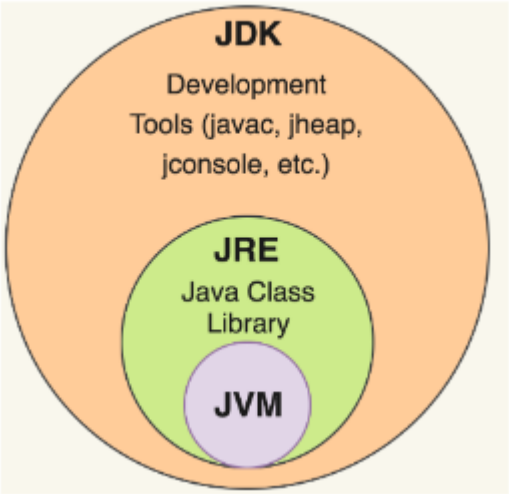
Difference between object and class

No.	Object	Class
1)	Object is an **instance** of a class.	Class is a **blueprint or template** from which objects

		are created.
2)	Object is a **real world entity** such as pen, laptop, mobile, bed, keyboard, mouse, chair etc.	Class is a **group of similar objects**.
3)	Object is a **physical** entity.	Class is a **logical** entity.
4)	Object is created through **new keyword** mainly e.g. Student s1=new Student();	Class is declared using **class keyword** e.g. class Student{}
5)	Object is created **many times** as per requirement.	Class is declared **once**.
6)	Object **allocates memory when it is created**.	Class **doesn't allocated memory when it is created**.
7)	There are **many ways to create object** in java such as new keyword, newInstance() method, clone() method, factory method and deserialization.	There is only **one way to define class** in java using class keyword.

Java Development Kit (JDK):

JDK contains tools needed ,To develop the Java programs and JRE to run the programs.
The tools include compiler (javac.exe),Java application launcher (java.exe), Appletviewer, etc...
Java application launcher (java.exe) opens a JRE, loads the class, and invokes its main method.



Java Runtime Environment (JRE)

The JRE is required to run java applications.

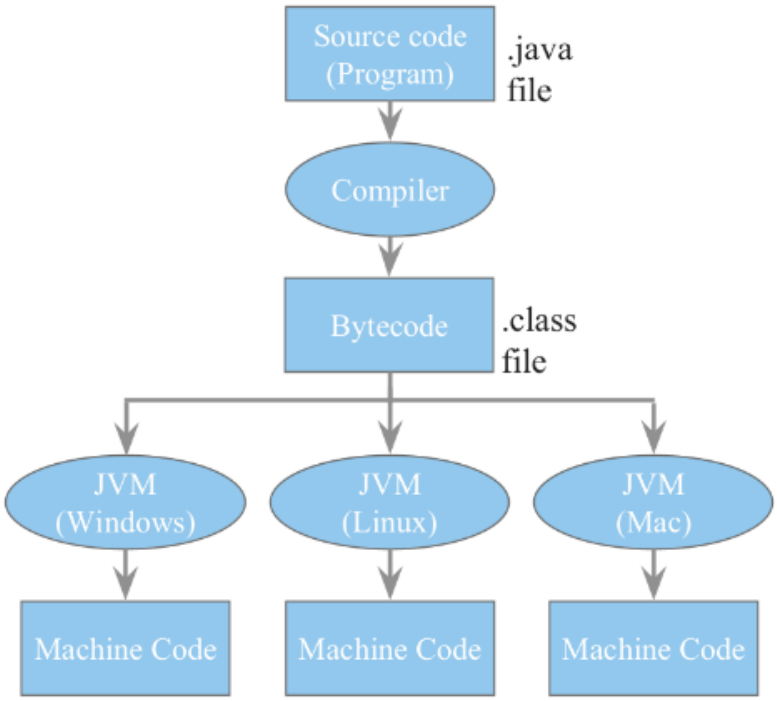
It combines the Java Virtual Machine (JVM), platform core classes and supporting libraries.

JRE is part of the Java Development Kit (JDK), but can be downloaded separately.

It does not contain any development tools such as compiler, debugger, etc.

Java Virtual Machine (JVM)

JVM is a virtual machine that enables a computer to run Java programs as well as programs written in other languages and compiled to Java Bytecode. Bytecode is a highly optimized set of instructions designed to be executed by the Java Virtual Machine(JVM). Byte code is intermediate representation of java source code. Java compiler provides byte code by compiling Java Source Code. Extension for java class file or byte code is '.class', which is platform independent. JVM is virtual because , It provides a machine interface that does not depend on the operating system and machine hardware architecture. JVM interprets the byte code into the machine code. JVM itself is platform dependent, but Java is Not.



A First Simple Program

```
public class HelloWorld {

   public static void main(String[] args) {
      System.out.println("Hello World");
   }

}
```

Output:

Hello World

Keywords

Java Keywords

Java keywords are also known as reserved words. Keywords are particular words that act as a key to a code. These are predefined words by Java so they cannot be used as a variable or object name or class name.

List of Java Keywords

A list of Java keywords or reserved words are given below:

1. **abstract**: Java abstract keyword is used to declare an abstract class. An abstract class can provide the implementation of the interface. It can have abstract and non-abstract methods.
2. **boolean**: Java boolean keyword is used to declare a variable as a boolean type. It can hold True and False values only.
3. **break**: Java break keyword is used to break the loop or switch statement. It breaks the current flow of the program at specified conditions.
4. **byte**: Java byte keyword is used to declare a variable that can hold 8-bit data values.
5. **case**: Java case keyword is used with the switch statements to mark blocks of text.
6. **catch**: Java catch keyword is used to catch the exceptions generated by try statements. It must be used after the try block only.
7. **char**: Java char keyword is used to declare a variable that can hold unsigned 16-bit Unicode characters
8. **class**: Java class keyword is used to declare a class.
9. **continue**: Java continue keyword is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition.
10. **default**: Java default keyword is used to specify the default block of code in a switch statement.
11. **do**: Java do keyword is used in the control statement to declare a loop. It can iterate a part of the program several times.
12. **double**: Java double keyword is used to declare a variable that can hold 64-bit floating-point number.
13. **else**: Java else keyword is used to indicate the alternative branches in an if statement.
14. **enum**: Java enum keyword is used to define a fixed set of constants. Enum constructors are always private or default.
15. **extends**: Java extends keyword is used to indicate that a class is derived from another class or interface.
16. **final**: Java final keyword is used to indicate that a variable holds a constant value. It is used with a variable. It is used to restrict the user from updating the value of the variable.
17. **finally**: Java finally keyword indicates a block of code in a try-catch structure. This block is always executed whether an exception is handled or not.
18. **float**: Java float keyword is used to declare a variable that can hold a 32-bit floating-point number.
19. **for**: Java for keyword is used to start a for loop. It is used to execute a set of instructions/functions repeatedly when some condition becomes true. If the number of iteration is fixed, it is recommended to use for loop.
20. **if**: Java if keyword tests the condition. It executes the if block if the condition is true.
21. **implements**: Java implements keyword is used to implement an interface.
22. **import**: Java import keyword makes classes and interfaces available and accessible to the current source code.
23. **instanceof**: Java instanceof keyword is used to test whether the object is an instance of the specified class or implements an interface.
24. **int**: Java int keyword is used to declare a variable that can hold a 32-bit signed integer.
25. **interface**: Java interface keyword is used to declare an interface. It can have only abstract methods.

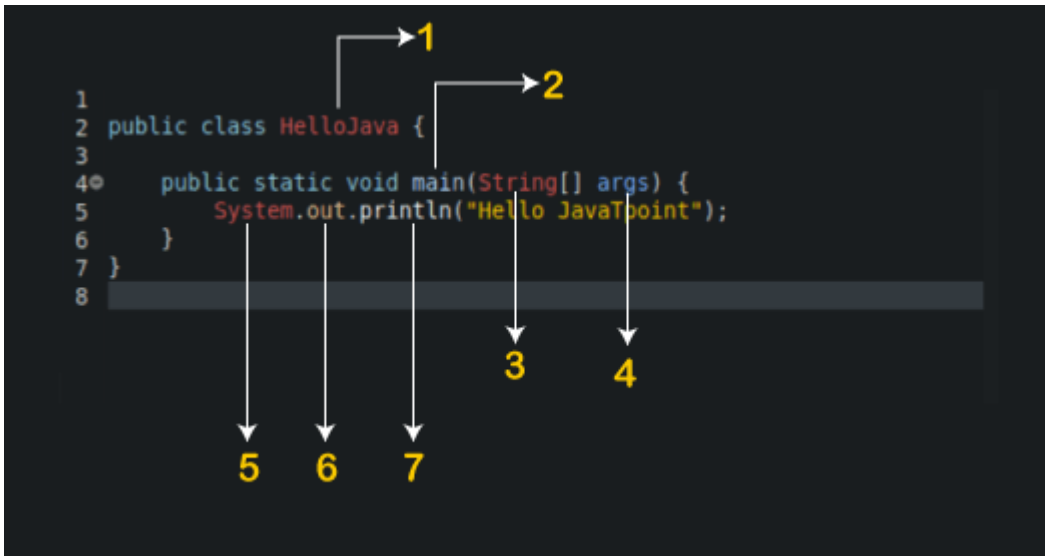
26. **long**: Java long keyword is used to declare a variable that can hold a 64-bit integer.
27. **native**: Java native keyword is used to specify that a method is implemented in native code using JNI (Java Native Interface).
28. **new**: Java new keyword is used to create new objects.
29. **null**: Java null keyword is used to indicate that a reference does not refer to anything. It removes the garbage value.
30. **package**: Java package keyword is used to declare a Java package that includes the classes.
31. **private**: Java private keyword is an access modifier. It is used to indicate that a method or variable may be accessed only in the class in which it is declared.
32. **protected**: Java protected keyword is an access modifier. It can be accessible within the package and outside the package but through inheritance only. It can't be applied with the class.
33. **public**: Java public keyword is an access modifier. It is used to indicate that an item is accessible anywhere. It has the widest scope among all other modifiers.
34. **return**: Java return keyword is used to return from a method when its execution is complete.
35. **short**: Java short keyword is used to declare a variable that can hold a 16-bit integer.
36. **static**: Java static keyword is used to indicate that a variable or method is a class method. The static keyword in Java is mainly used for memory management.
37. **strictfp**: Java strictfp is used to restrict the floating-point calculations to ensure portability.
38. **super**: Java super keyword is a reference variable that is used to refer to parent class objects. It can be used to invoke the immediate parent class method.
39. **switch**: The Java switch keyword contains a switch statement that executes code based on test value. The switch statement tests the equality of a variable against multiple values.
40. **synchronized**: Java synchronized keyword is used to specify the critical sections or methods in multithreaded code.
41. **this**: Java this keyword can be used to refer the current object in a method or constructor.
42. **throw**: The Java throw keyword is used to explicitly throw an exception. The throw keyword is mainly used to throw custom exceptions. It is followed by an instance.
43. **throws**: The Java throws keyword is used to declare an exception. Checked exceptions can be propagated with throws.
44. **transient**: Java transient keyword is used in serialization. If you define any data member as transient, it will not be serialized.
45. **try**: Java try keyword is used to start a block of code that will be tested for exceptions. The try block must be followed by either catch or finally block.
46. **void**: Java void keyword is used to specify that a method does not have a return value.
47. **volatile**: Java volatile keyword is used to indicate that a variable may change asynchronously.
48. **while**: Java while keyword is used to start a while loop. This loop iterates a part of the program several times. If the number of iteration is not fixed, it is recommended to use the while loop.

Identifiers

Identifiers in Java are symbolic names used for identification. They can be a class name, variable name, method name, package name, constant name, and more. However, In Java, There are some reserved words that can not be used as an identifier.

For every identifier there are some conventions that should be used before declaring them. Let's understand it with a simple Java program:

```
.        public class HelloJava {
.            public static void main(String[] args) {
.                System.out.println("Hello JavaTpoint");
.            }
.        }
```



1. HelloJava (Class name)
2. main (main method)
3. String (Predefined Class name)
4. args (String variables)
5. System (Predefined class)
6. out (Variable name)

7. println (method)

Java Top 10 Libraries

Java is one of the most popular programming languages. Java provides a rich set of libraries, and its standard Java library is a very powerful that contains libraries such as **java.lang**, **java.util**, and **java.math**, etc. Java provides more than thousands of libraries except standard libraries. Some of the most useful and popular libraries are as follows:

1. Java Standard libraries
2. Apache Commons
3. Jackson
4. Maven
5. Google-json
6. Log5j and Slf4j
7. JUnit
8. Google Guava
9. JAXB
10. HTTP Libraries